Appendix C | Written solutions

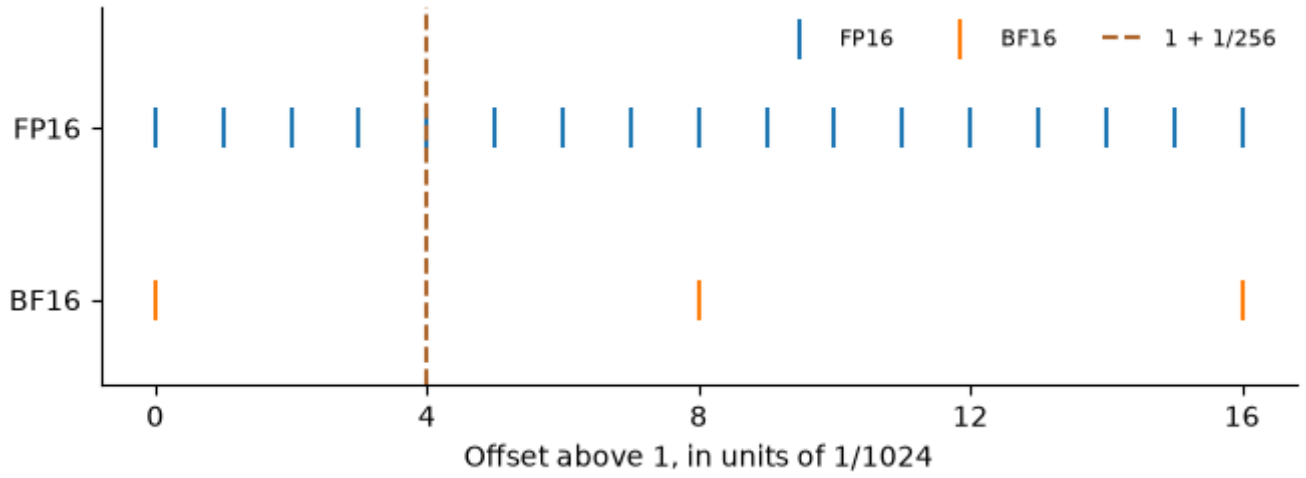
Stable Numerical Computation

Stable formulas preserve useful answers even when an intermediate probability underflows. Separate the information lost when inputs are stored from the error introduced by subsequent arithmetic; a wider accumulator can address only the latter.

Eight exercises progress from tracing small calculations to diagnosing implementation errors. These are controlled numerical examples, not performance or model-quality benchmarks.

1. Compare representable gaps

Problem. Under round-to-nearest, what happens to $1 + 2^{-8}$ in FP16 and in BF16 with ties to even?
Solution. The offset $2^{-8} = 1/256$ equals four FP16 gaps of $1/1024$, so FP16 represents it exactly: 1.00390625. A BF16 gap at 1 is $1/128$, so the number is exactly halfway between 1 and $1 + 1/128$. The lower endpoint has an even last significand bit, so ties-to-even chooses 1. More exponent range does not imply more precision near one.



2. Shift before exponentiating

Problem. For logits $(1000, 1000 + \log 2, 1000 + \log 5)$, give shifted exponentials, probabilities, and the middle-class loss.
Solution. The maximum is $1000 + \log 5$. Subtracting it yields exponentials $(1/5, 2/5, 1)$ with sum $8/5$. The probabilities are $(1/8, 2/8, 5/8)$. The middle-class loss is $-\log(2/8) = \log 4$, approximately 1.386294. Subtracting a shared offset preserves differences and avoids overflowing the exponentials.

3. Normalize only permitted keys

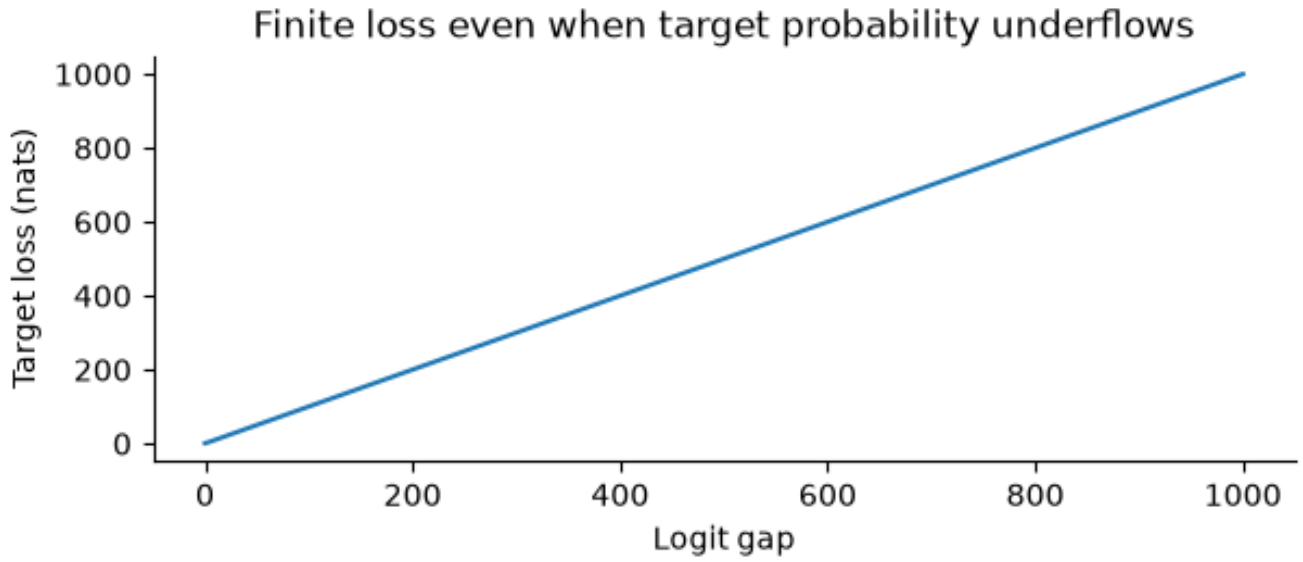
Problem. Normalize scores $(4, 1, 3)$ with only the last two keys allowed. Then remove every allowed key. What does the canonical attention routine specify?
Solution. The excluded score 4 does not participate in either the maximum or denominator. The permitted maximum is 3, so shifted contributions are $(0, e^{-2}, 1)$. Weights are $(0, 1/(1 + e^2), e^2/(1 + e^2))$, approximately $(0, 0.119203, 0.880797)$. An entirely masked row raises ValueError in the book implementation. It does not claim that an empty set defines a probability distribution.

4. Audit low-precision normalization

Problem. A proposed RMSNorm uses FP16 inputs, squares, and output. Choose a small input, a high-precision reference, and intermediates to inspect. What should change if squares become zero?
Solution. Use $(10^{-4}, -10^{-4})$, unit gains, and epsilon 10^{-6}. First inspect the actual stored inputs, then their squares, mean square, denominator, normalized values, and output conversion. The FP16 squares round to zero, losing a mean square near 10^{-8}. Cast inputs before squaring and accumulate in FP32. For an arithmetic check, form the float64 reference from the stored inputs rather than the original decimal values. The canonical implementation does this wider calculation and then rounds its output back to the input dtype. A small explicit tolerance covers that final rounding.

5. Keep the log loss finite

Problem. For logits $(0, -1000)$ and the second class as target, compare the rounded probability with the stable loss. Add 2000 to both logits and repeat.
Solution. The exponential of -1000 underflows to zero in float64, so the stored target probability is zero. Taking its logarithm would give negative infinity and an infinite loss. Directly from logits, the loss is $\log(1 + e^{-1000}) + 1000$, which rounds to 1000. After the common shift, logits $(2000, 1000)$ have exactly the same difference and the same stable loss. This avoids a damaging intermediate computation rather than changing the mathematical objective.



6. Locate lost input information

Problem. Store logits $(1024, 1024.25)$ in FP16. Compare softmax on the original float64 values, float64 on the stored values, and low-precision computation. Which discrepancy can a wider accumulator fix?
Solution. At 1024 the FP16 gap is 1, so both stored logits become 1024. Their float64 softmax is therefore $(0.5, 0.5)$, as is the low-precision result. The original logits give approximately $(0.4378235, 0.5621765)$. That discrepancy is input quantization error: the difference 0.25 has already disappeared. A wider accumulator can reduce later arithmetic error but cannot infer the lost logit difference. Here the computation on the stored values is exact, so no accumulator improvement can restore the original probabilities.

7. Place epsilon deliberately

Problem. Normalize $(10^{-4}, -10^{-4})$ with epsilon 10^{-6} inside and outside the root. Check zero inputs. Why is the zero test insufficient?
Solution. The mean square is $q = 10^{-8}$. The inside-root denominator is $\sqrt{1.01 \times 10^{-6}} \approx 0.00100499$, giving magnitudes about 0.0995037. The outside-root denominator is $\sqrt{10^{-8}} + 10^{-6} = 0.000101$, giving about 0.990099. Both return zeros on zero inputs because both denominators are positive. Passing that one boundary test does not make the two functions equivalent; small nonzero inputs expose their different scaling.

8. Use absolute and relative tolerances

Problem. Reference values $(0, 10^{-8}, 1000)$ are compared with $(10^{-9}, 1.1 \times 10^{-8}, 1000.01)$ using atol 2×10^{-9} and rtol 2×10^{-5}. Evaluate combined, absolute-only, and relative-only checks; check NaNs separately.
Solution. Absolute errors are approximately $(10^{-9}, 10^{-9}, 0.01)$. Combined allowances are $(2 \times 10^{-9}, 2.0002 \times 10^{-9}, 0.020000002)$, so all pass. Absolute-only tolerances pass the first two but fail the last. Relative-only tolerances have allowances $(0, 2 \times 10^{-13}, 0.02)$, so the first two fail and the last passes. The absolute term protects small reference values. Check finiteness before comparing errors; never treat NaNs as valid matching values merely to make a test pass.